



FIG. 1 : Un chat roux assis à côté d'une marionnette en bois de Pinocchio sur le rebord d'une fenêtre ouverte donnant sur un jardin — le nez allongé de Pinocchio évoque le mensonge, en écho à la question posée par le titre de ce document (image générée par IA, Gemini)

Table des matières

Mistral est-il vraiment open source ? **2**

Introduction 2

1. Logiciel libre et standards ouverts : les fondamentaux 2

 Les quatre libertés du logiciel libre 2

 Standards ouverts : une notion distincte 3

2. Le continuum d'ouverture en intelligence artificielle 3

 Pourquoi l'IA complique la notion d'open source 3

 La définition OSAID : qui décide ce qu'est l'open source en IA ? 3

 Open weight vs open source : la distinction qui change tout 5

 Pourquoi cette distinction importe vraiment 5

 Les quatre catégories du continuum 5

 Ce que le continuum change concrètement 6

3. Réglementation et frugalité : deux angles souvent oubliés 7

 Ce que dit l'AI Act 7

 Trois définitions qui ne se recouvrent pas 7

 La fausse bonne idée : local est-il vraiment plus écologique ? 8

 Le paradoxe de Jevons 9

4. Ollama, la conclusion, et un dernier rebondissement 9

 Ollama : l'ouverture en pratique 9

 Alors, Mistral est-il vraiment open source ? 9

 Le vrai dilemme : Ollama ou Infomaniak 9

Sources et vérifications 10

 Point de départ de la veille 10

 Open Source AI Definition (OSAID) et logiciel libre 10

 AI Act et réglementation 11

 Frugalité énergétique et efficacité de l'inférence 11

 Ollama — tarification et modèles 11

 Infomaniak — alternative souveraine 11

 Modèles d'IA mentionnés, par catégorie du continuum 11

Mistral est-il vraiment open source ?

Décortiquer le continuum d'ouverture en intelligence artificielle

Introduction

Llama, Mistral, DeepSeek : ces modèles sont régulièrement qualifiés d'« open source » dans la presse comme dans le marketing des entreprises qui les publient. Ce document démontre que ce vocabulaire est le plus souvent inexact, et explique pourquoi la distinction n'est pas qu'une question de vocabulaire administratif.

Pour y répondre, il faut d'abord revenir aux fondamentaux du logiciel libre, comprendre pourquoi l'intelligence artificielle complique cette notion plus qu'un logiciel classique, puis regarder ce que dit la réglementation européenne et ce qu'implique le choix entre exécuter un modèle chez soi ou via une API distante. La conclusion réserve un dernier rebondissement, qui montre que même la réponse la plus honnête cache encore un compromis.

1. Logiciel libre et standards ouverts : les fondamentaux

Les quatre libertés du logiciel libre

La Free Software Foundation formalise depuis 1986 ce qu'est un logiciel libre autour de quatre libertés fondamentales, numérotées de 0 à 3 :

- **Liberté 0 — Utiliser** : pouvoir exécuter le programme comme on le souhaite, pour n'importe quel usage (personnel, commercial, militaire, tout est permis).
- **Liberté 1 — Étudier et modifier** : pouvoir comprendre comment fonctionne le programme et le modifier. Cette liberté suppose l'accès au code source, pas seulement au binaire compilé.
- **Liberté 2 — Redistribuer** : pouvoir partager des copies du programme avec d'autres.

- **Liberté 3 — Redistribuer ses modifications** : pouvoir modifier le programme puis partager cette nouvelle version avec d'autres.

Une image simple aide à retenir ces quatre libertés : c'est comme une recette de cuisine. La liberté 0, c'est pouvoir faire le plat. La liberté 1, c'est pouvoir lire la recette et la modifier — ajouter du sel, changer les quantités. La liberté 2, c'est donner la recette à des amis. La liberté 3, c'est donner sa propre version modifiée de la recette. Ce qui compte n'est donc pas seulement de pouvoir utiliser un logiciel, mais aussi de pouvoir le modifier et le partager.

Une clarification s'impose : **libre ne veut pas dire gratuit**. C'est une question de libertés d'action, pas de prix. Un logiciel libre peut parfaitement être vendu, utilisé commercialement ou intégré à une activité économique rémunérée.

Standards ouverts : une notion distincte

Les **standards ouverts** — des spécifications documentées, souvent publiées sous forme de RFC (Request for Comments) — permettent à des systèmes différents d'interagir. TCP/IP, HTTP, HTML ou CSS en sont des exemples : le Web fonctionne parce que tout le monde respecte les mêmes spécifications techniques.

Le port USB illustre bien cette idée. Peu importe qu'on branche un iPhone, un téléphone Samsung, un ordinateur Windows ou un Mac : tant que l'appareil respecte le standard USB, la connexion fonctionne. Le matériel derrière la prise n'a aucune importance — seul compte le respect du standard. Avec un standard ouvert, on ne demande jamais « comment fonctionne ce produit en interne ? » (une question souvent sans réponse, le produit restant fermé), mais seulement « respecte-t-il le standard ? ». Si oui, l'interopérabilité est acquise.

Cette notion est différente de celle du logiciel libre : un standard ouvert apporte la **liberté d'interagir et de changer de fournisseur sans verrouillage**, pas nécessairement la liberté de modifier le code d'une implémentation particulière de ce standard.

2. Le continuum d'ouverture en intelligence artificielle

Pourquoi l'IA complique la notion d'open source

Pour un logiciel traditionnel, le code source constitue l'élément central qui permet de comprendre comment le programme fonctionne : un seul artefact concentre toute l'information nécessaire.

Un modèle de machine learning est radicalement différent. Son comportement ne provient pas uniquement du programme qui l'exécute, mais résulte de **trois composantes distinctes** :

1. **Les poids du modèle** : les paramètres numériques appris pendant l'entraînement.
2. **Le code et le processus d'entraînement** : architecture, prétraitement, hyperparamètres, algorithmes.
3. **Les données d'entraînement** : le corpus sur lequel le modèle a appris.

Chacune de ces trois composantes peut être ouverte, fermée ou partiellement publiée **indépendamment des deux autres**. C'est cette multiplication de degrés d'ouverture qui crée un continuum, là où le logiciel classique ne connaît qu'une alternative binaire (code source disponible ou non).

L'analogie d'un gâteau aide à visualiser ces trois composantes. Les poids correspondent au gâteau fini — le résultat qu'on peut consommer. Le code et le processus d'entraînement correspondent à la recette — la température, la durée, l'ordre des étapes. Les données correspondent aux ingrédients — la farine, les œufs, et leur qualité. Si l'on dispose uniquement du gâteau fini (autrement dit des poids seuls), on peut le manger, mais on ne sait ni comment il a été fait, ni le reproduire à l'identique.

La définition OSAID : qui décide ce qu'est l'open source en IA ?

L'**Open Source Initiative (OSI)**, qui fait autorité sur la définition du logiciel libre depuis 1998, a développé en 2024 l'**Open Source AI Definition (OSAID)**, qui applique les quatre libertés du logiciel libre à l'intelligence artificielle.

Pour être certifié « open source » au sens OSAID, un modèle doit permettre :

1. **Utiliser librement** (liberté 0).

POIDS
=
Le gâteau fini

RECETTE
=
Le processus (température, durée, ordre)

DONNÉES
=
Les ingrédients (farine, œufs, qualité)

FIG. 2 : Trois boîtes illustrant la correspondance entre les composantes d'un modèle d'IA et l'analogie du gâteau : Poids = le gâteau fini, Recette = le processus d'entraînement (température, durée, ordre), Données = les ingrédients (farine, œufs, qualité)

2. **Étudier et comprendre** comment fonctionne le modèle (liberté 1) — ce qui nécessite l'accès aux poids, aux données et à la recette d'entraînement.
3. **Modifier et réentraîner** (liberté 2) — impossible si les données ou le code d'entraînement manquent.
4. **Redistribuer** les modifications (liberté 3).

Open weight vs open source : la distinction qui change tout

Cette exigence entraîne une conséquence majeure. **Open weight** désigne les modèles dont les poids sont publiés, mais dont les données et le code d'entraînement restent fermés : on peut utiliser le modèle, mais pas réellement l'étudier ni le modifier (les libertés 1 à 3 sont bloquées). Llama, Mistral et DeepSeek relèvent de cette catégorie.

Open source, au sens strict de l'OSAID, désigne les modèles dont les poids, les données et le code sont tous publiés : on peut alors utiliser, étudier, modifier et redistribuer. OLMo, Amber, Luciole ou Nemotron relèvent de cette catégorie.

C'est cette distinction qui répond directement à la question posée par le titre de ce document : **Mistral n'est pas open source, il est open weight**. Pour être open source au sens strict, il faudrait aussi publier les données et le code d'entraînement.

L'image du gâteau permet ici aussi de saisir l'enjeu. Être open weight, c'est montrer le gâteau fini avec un emballage soigné, sans donner la recette ni la liste des ingrédients : impossible de vérifier s'il contient un allergène ou un biais caché. Être open source, c'est montrer le gâteau, la recette complète, et la liste exacte des ingrédients : on peut utiliser, modifier, refaire, vérifier. L'open weight repose sur une **confiance aveugle** envers celui qui a fabriqué le modèle ; l'open source permet une **confiance vérifiée**, où l'audit reste toujours possible.

Pourquoi cette distinction importe vraiment

Ce n'est pas une question administrative de cases cochées. Deux exemples concrets montrent ce qui est réellement en jeu.

Premier exemple — les données cachées créent des biais invisibles. Imaginons un modèle entraîné sur un corpus contaminé par la propagande d'un régime autoritaire : le modèle en hérite les biais, sans que cela soit visible de l'extérieur. Avec l'open weight seul, on peut constater qu'une réponse du modèle semble étrange, sans jamais pouvoir en identifier l'origine — impossible même de l'auditer. Avec l'open source, l'accès aux données d'entraînement permet de voir précisément d'où vient le biais, et donc de le corriger. Reprise dans les termes de l'analogie du gâteau : c'est comme une eau contaminée invisible utilisée dans la préparation. Avec l'open weight, on goûte un résultat bizarre sans pouvoir en identifier la cause ; avec l'open source, la recette et les ingrédients permettent de repérer immédiatement la source contaminée. L'indépendance de penser dépend donc directement de la transparence de la fabrication.

Second exemple — les erreurs méthodologiques révélées par l'ouverture. En janvier 2025, le modèle français Lucie (développé par l'association OpenLLM France) a été lancé dans une version inachevée, non censurée, sans garde-fous suffisants. Ses réponses étaient parfois incohérentes, avec des erreurs flagrantes. Parce que le projet était ouvert, la communauté a pu détecter ces problèmes et les signaler publiquement, ce qui a forcé une correction rapide. Si Lucie avait été un modèle fermé, personne n'aurait jamais su que ces défauts méthodologiques existaient. L'image est celle d'un restaurant qui ferme momentanément pour refaire sa cuisine après la découverte de bactéries : dans un système fermé, les clients n'auraient jamais su qu'il y avait un problème ; dans un système ouvert, tout le monde a pu voir le problème, ce qui a permis la correction. L'ouverture permet donc la correction communautaire.

Les quatre catégories du continuum

Le continuum d'ouverture des modèles d'IA se répartit en quatre catégories, allant de la fermeture complète à l'ouverture totale.

Fermé / API. Aucune composante n'est accessible : ni les poids, ni le code, ni les données. Seul un accès distant via une API est proposé, parfois avec des restrictions d'usage commercial. La liberté de l'utilisateur se limite à l'exécution, via la plateforme du fournisseur. GPT (OpenAI), Claude (Anthropic), Gemini (Google) ou Amazon Nova relèvent de cette catégorie. Cette fermeture n'est pas nécessairement critiquable en soi : c'est un choix stratégique valide, ces entreprises investissant massivement dans la recherche et développement, et choisissant de monétiser via l'infrastructure cloud plutôt que de laisser circuler les poids.

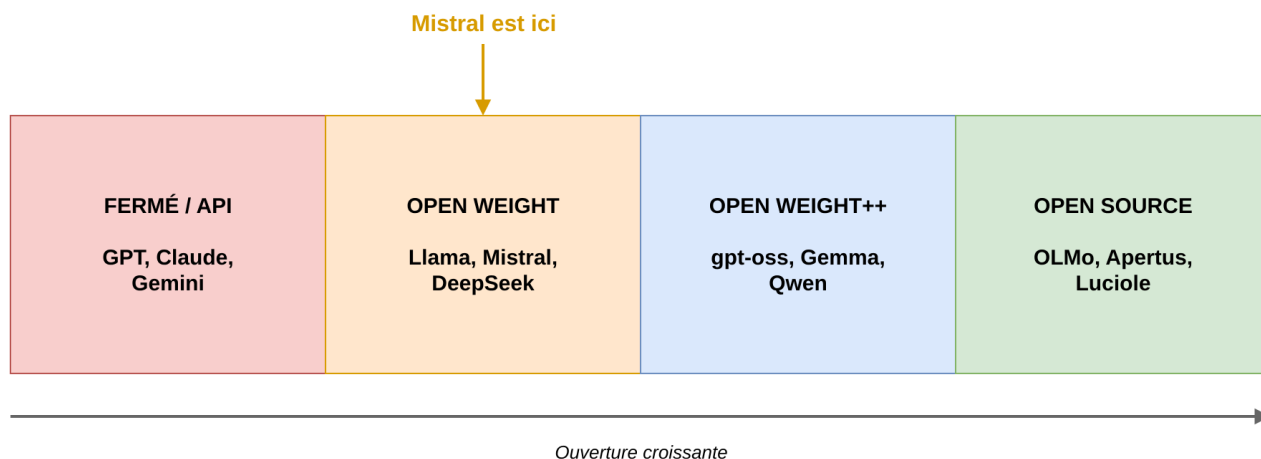


FIG. 3 : Schéma en gradient représentant les quatre catégories du continuum d'ouverture, de gauche à droite : Fermé/API (GPT, Claude, Gemini), Open weight (Llama, Mistral, DeepSeek — Mistral est positionné ici), Open weight++ (gpt-oss, Gemma, Qwen), Open source (OLMo, Apertus, Luciole), avec une flèche indiquant une ouverture croissante

Open weight. Les poids sont publiés, souvent sous une licence permissive de type Apache 2.0 ou MIT, mais ni le code ni les données ne le sont. L'utilisateur peut télécharger et exécuter le modèle localement (liberté 0), mais ne peut pas réellement l'étudier ni le modifier de façon éclairée (libertés 1 à 3 bloquées, faute d'information suffisante). Llama, Mistral et DeepSeek relèvent de cette catégorie. Bien que très populaires et largement documentés, ils entrent techniquement dans cette catégorie intermédiaire, et non dans celle de l'open source.

Open weight++. Une catégorie intermédiaire, où les poids sont publiés avec une documentation significative du processus d'entraînement, sans toutefois donner un accès complet aux données ni un code d'entraînement réellement reproductible. gpt-oss, Gemma ou Qwen relèvent de cette catégorie : suffisamment transparente pour inspirer confiance, pas assez pour permettre une reproduction indépendante complète. On peut étudier davantage que dans la catégorie précédente, mais toujours pas modifier vraiment, faute de données complètes ou d'outils de réentraînement.

Open source (au sens strict de l'OSAID). Les poids, le code et les données sont tous publiés et librement modifiables, avec des scripts d'entraînement reproductibles. C'est la seule catégorie où les quatre libertés du logiciel libre s'appliquent réellement. OLMo (Allen Institute for AI), Apertus (LLM360) et Luciole (OpenLLM France) en font partie. Les acteurs de cette catégorie sont académiques (AI2, EleutherAI, LLM360), associatifs (OpenLLM France), mais aussi parfois industriels — NVIDIA, avec son modèle Nemotron, démontre que l'ouverture complète n'est pas l'apanage des seuls petits acteurs.

Ce que le continuum change concrètement

Cette distinction a des conséquences pratiques très différentes selon le profil de l'utilisateur.

Pour un **chercheur en reproductibilité**, un modèle fermé/API rend impossible tout audit ou test de robustesse. Un modèle open weight permet de tester le comportement du modèle, sans comprendre comment il a été entraîné. Un modèle open weight++ offre une meilleure documentation, sans permettre de refaire l'entraînement. Seul un modèle open source permet d'auditer, reproduire et modifier intégralement.

Pour une **entreprise cherchant l'indépendance technologique**, un modèle fermé/API crée une dépendance totale au fournisseur (coût, disponibilité, conformité réglementaire). Un modèle open weight permet l'auto-hébergement, mais rend le fine-tuning sérieux difficile. Un modèle open weight++ facilite l'adaptation grâce à une meilleure documentation. Seul un modèle open source garantit une réelle indépendance, avec la maîtrise du cycle complet.

Pour une **contribution communautaire**, un modèle fermé/API n'autorise aucune contribution. Un modèle open weight permet de créer des outils d'inférence, sans améliorer le modèle lui-même. Un modèle open weight++ autorise des propositions d'adaptation. Seul un modèle open source permet de contribuer directement au modèle de base.

Le message central de cette section peut se résumer ainsi : le continuum montre que « ouvert » en intelligence artificielle ne signifie pas une seule chose. Publier les poids n’est pas la même chose que publier le code et les données. C’est exactement pour cette raison que Llama et Mistral — bien que leurs poids soient accessibles — ne satisfont pas la définition « open source » au sens de la Free Software Foundation ou de l’OSAIID. Dire « open source » à leur sujet est donc inexact ; dire « open weight » est plus précis. Pour de l’open source au sens historique du terme, il faut aller jusqu’à la quatrième catégorie du continuum.

3. Réglementation et frugalité : deux angles souvent oubliés

Une fois le continuum technique posé, une question réglementaire se pose naturellement : comment les gouvernements définissent-ils « open source » pour l’intelligence artificielle ? L’Union européenne, avec l’AI Act, a apporté une réponse qui n’est pas celle qu’on attendrait spontanément.

Ce que dit l’AI Act

L’AI Act européen (règlement 2024/1689) **exempte** les modèles open source des règles de risque systémique les plus contraignantes (article 2(12)). La raison avancée est que, si le code et les poids sont transparents, la communauté peut auditer et corriger les problèmes, ce qui réduit le risque d’une défaillance centralisée non détectée.

Mais cette exemption connaît une limite : elle ne s’applique pas si le modèle dépasse un seuil de puissance de calcul de 10^{25} FLOPs — environ dix milliards de milliards d’opérations (article 51(2)). Le cas de Llama-3 405B illustre ce paradoxe : ce modèle atteint environ $3,8 \times 10^{25}$ FLOPs, donc au-dessus du seuil. Même s’il était considéré comme open source, il n’échapperait pas aux règles de risque systémique renforcées.

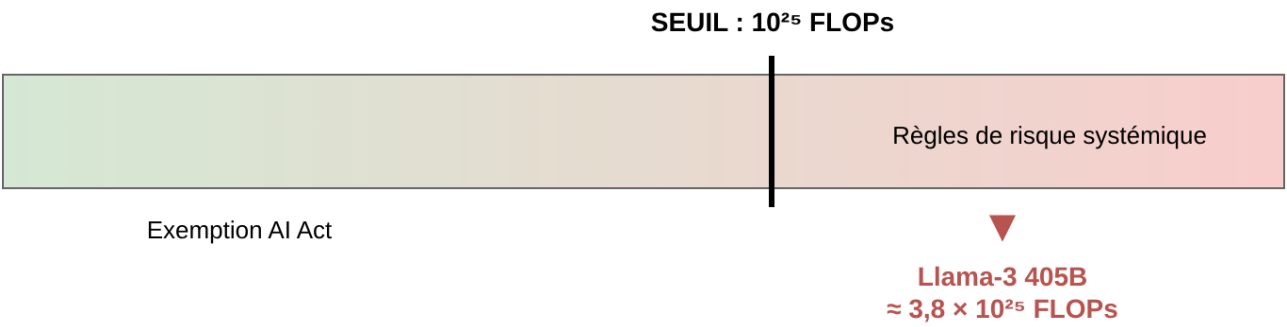


FIG. 4 : Jauge illustrant le seuil de 10^{25} FLOPs de l’article 51(2) de l’AI Act : en dessous, exemption pour les modèles open source ; au-dessus, application des règles de risque systémique quel que soit le statut d’ouverture. Llama-3 405B (environ $3,8 \times 10^{25}$ FLOPs) est positionné au-dessus du seuil

Trois définitions qui ne se recouvrent pas

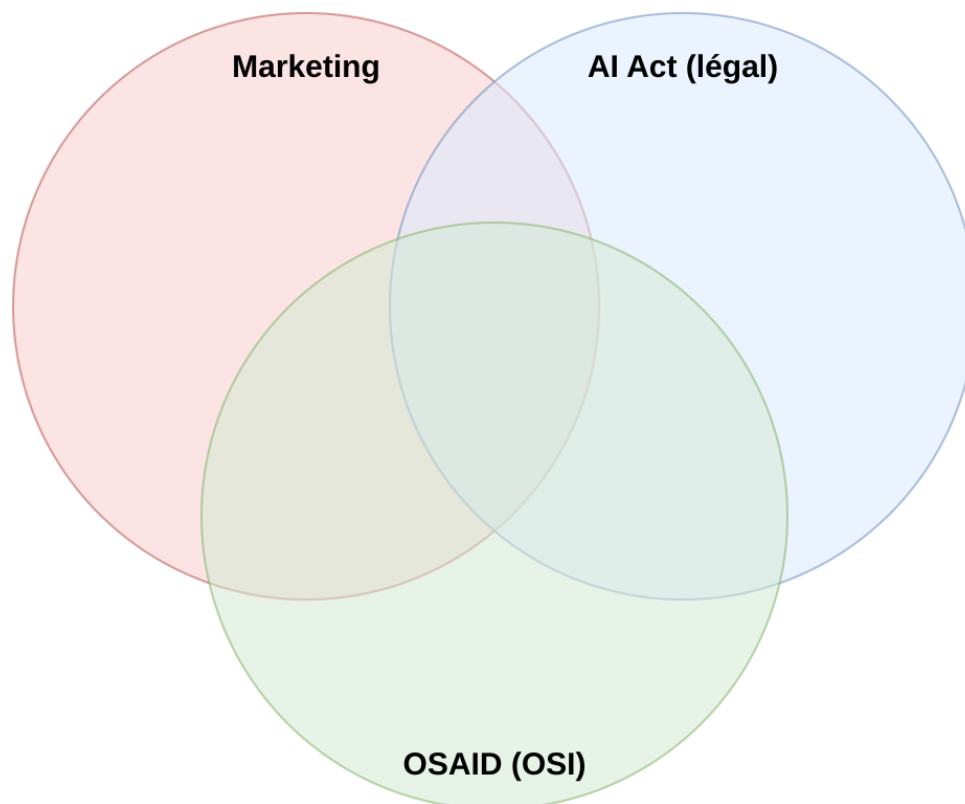
Ce détour réglementaire révèle un problème plus large : le mot « open source » recouvre en réalité **trois définitions distinctes**, qui ne coïncident pas exactement.

Contexte	Ce que « open source » signifie	Ce que ça exige des données d’entraînement
Marketing commercial	Poids publiés, transparence partielle	Rien
AI Act (cadre légal)	Licence libre, paramètres et architecture publics, modèle non monétisé	Un résumé du contenu d’entraînement (modèle officiel de la Commission), pas les données elles-mêmes
OSAIID (OSI)	Utiliser, étudier, modifier, redistribuer	Des informations suffisantes pour reconstruire un système équivalent

Contrairement à ce qu'on pourrait supposer, la définition de l'AI Act est **plus légère que celle de l'OSAIID**, mais elle n'est pas vide pour autant. Le régime allégé dispense de la documentation technique détaillée (article 53(1)(a-b)) et de la désignation d'un représentant dans l'Union (article 54) ; en revanche, deux obligations subsistent même pour un modèle publié en open source : mettre en place une politique de respect du droit d'auteur (article 53(1)(c)) et publier un résumé détaillé du contenu d'entraînement (article 53(1)(d)). L'AI Act demande donc de **dire** sur quoi le modèle a été entraîné, sans exiger de **publier** les données — là où l'OSAIID exige un niveau de détail permettant la reconstruction.

Deux conditions de l'AI Act méritent d'être soulignées, car elles sont souvent oubliées dans les discussions : le modèle ne doit pas être **monétisé** (considérant 103 : les composants fournis contre paiement ou monétisés d'une autre façon ne bénéficient pas de ces exceptions), et l'exemption devient de toute façon inopérante au-delà du seuil de risque systémique vu plus haut. Savoir si un modèle donné remplit réellement ces conditions relève de l'analyse juridique au cas par cas, pas d'une étiquette affichée par son éditeur : la licence Llama, par exemple, impose des restrictions d'usage qui posent question au regard de la notion de licence libre, et un éditeur qui commercialise l'accès à son modèle sort du cadre de l'exemption.

Aucune de ces trois définitions ne recouvre donc exactement les deux autres, ce qui explique une bonne partie de la confusion entretenue autour de ce vocabulaire — et ce qui fait de la question posée par le titre de ce document une vraie question à trois niveaux de réponse, et non une question rhétorique.



Données d'entraînement — Marketing : rien | AI Act : un résumé du contenu | OSAID : de quoi reconstruire

FIG. 5 : Diagramme de Venn à trois cercles partiellement chevauchés représentant les trois définitions de « open source » (Marketing, AI Act, OSAID) : aucun des trois cercles ne recouvre entièrement les deux autres, ce qui montre visuellement que ces définitions se recoupent partiellement sans jamais coïncider

La fausse bonne idée : local est-il vraiment plus écologique ?

Un second angle, souvent négligé dans les discussions sur l'ouverture des modèles d'IA, concerne l'impact environnemental. L'intuition répandue est la suivante : faire tourner un modèle chez soi serait forcément plus écologique — et moins coûteux — qu'une requête envoyée à une API cloud distante. La réalité est plus nuancée.

Un seuil déterminant se situe autour de **85 % d'utilisation du GPU**. En dessous de ce seuil, mieux vaut généralement faire tourner le modèle via une API cloud, grâce au *batching* (traitement de plusieurs requêtes simultanément, 8 à 20 à la fois plutôt qu'une seule), à un taux d'utilisation du matériel proche de la saturation

24 heures sur 24 (contre quelques heures par semaine pour un usage personnel), et à un PUE (*Power Usage Effectiveness*, l'indicateur d'efficacité énergétique d'un datacenter) compris entre 1,05 et 1,40, signe d'une faible surcharge énergétique liée à l'infrastructure. Au-dessus de ce seuil d'utilisation, un déploiement local devient compétitif.



FIG. 6 : Jauge illustrant le seuil de 85 % d'utilisation GPU : en dessous, le cloud est préférable (batching, PUE optimisé) ; au-dessus, le local devient compétitif

Le paradoxe de Jevons

Un exemple chiffré illustre la portée de ce sujet : GPT-4o consommerait environ 0,43 Wh par requête. À raison de 700 millions de requêtes par jour, cette seule consommation équivaldrait à l'alimentation électrique de 35 000 foyers américains. C'est une illustration du **paradoxe de Jevons** : lorsqu'une technologie devient plus efficace, elle est utilisée davantage, si bien que la consommation totale continue d'augmenter malgré les gains d'efficacité individuels.

La conclusion pragmatique de cette section est double : un déploiement local n'est pas forcément meilleur (sauf en cas d'utilisation vraiment intensive), et une API cloud n'est pas forcément pire (si le fournisseur optimise réellement son infrastructure). La vigilance de rigueur consiste à demander aux fournisseurs (OpenAI, Google, Anthropic, ou tout autre) leurs chiffres réels de PUE, plutôt que de supposer une réponse par principe. C'est précisément ce qui rend le choix d'un modèle open weight intéressant : il permet de décider, au cas par cas, entre un déploiement local et un déploiement cloud, selon l'usage réel.

La régulation (AI Act) et l'écologie (frugalité, paradoxe de Jevons) montrent ensemble que « open source » n'est pas qu'une question éthique abstraite : c'est une question d'indépendance, de transparence, et de choix réel.

4. Ollama, la conclusion, et un dernier rebondissement

Ollama : l'ouverture en pratique

Ollama offre une illustration concrète, déjà en production, de ce continuum d'ouverture. Son catalogue est exclusivement composé de modèles open weight et open source — aucun modèle fermé/API n'y figure. La plateforme elle-même est distribuée sous licence MIT : sa monétisation repose sur l'infrastructure cloud proposée en complément, pas sur les modèles.

Trois façons d'utiliser Ollama coexistent : en local, via une interface en ligne de commande gratuite (le code de cette interface étant lui-même sous licence MIT) ; via une API gratuite mais limitée ; ou via une offre Cloud Pro payante (20 dollars par mois, avec des crédits mensuels inclus, et des tarifs qui varient selon les heures de forte affluence).

Alors, Mistral est-il vraiment open source ?

La réponse directe est non : **Mistral est open weight**, pas open source. Pour l'être réellement, il faudrait aussi publier les données et la recette d'entraînement complète. « Open source » n'est pas une case que l'on coche ou non : c'est un continuum, et surtout, c'est un choix — celui de l'entreprise qui publie le modèle, mais aussi celui de l'utilisateur qui décide quel niveau de transparence lui est réellement nécessaire.

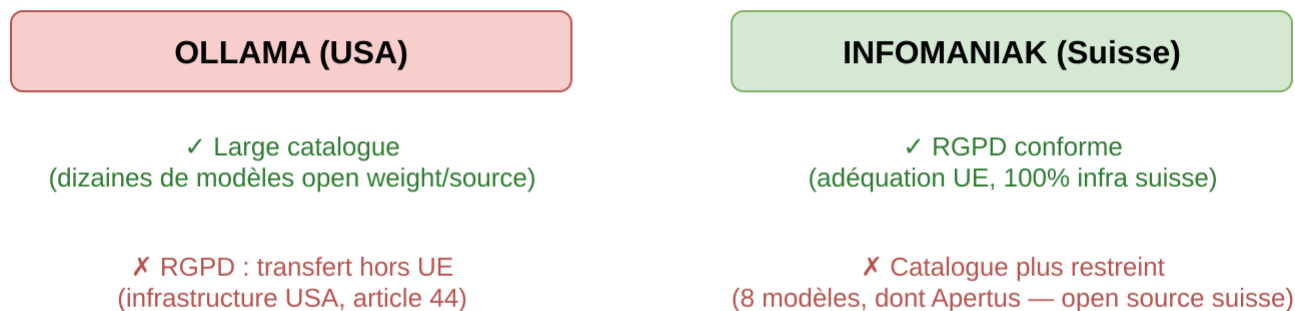
Le vrai dilemme : Ollama ou Infomaniak

Cette réponse, aussi honnête soit-elle, cache encore un dernier rebondissement. On pourrait se dire : « J'utiliserai Ollama Cloud, c'est de l'open source, donc je suis indépendant. » La réalité est plus compliquée. L'infrastructure

d'Ollama Cloud est hébergée aux États-Unis, ce qui constitue un transfert de données hors Union européenne au sens de l'article 44 du RGPD.

Une alternative existe : Infomaniak, hébergeur basé en Suisse, propose une infrastructure 100 % suisse et conforme au RGPD — la Suisse bénéficiant d'une décision d'adéquation de la Commission européenne, qui dispense de clauses contractuelles supplémentaires pour les transferts de données entre l'Union européenne et la Suisse. Mais son catalogue de modèles est plus restreint : huit modèles de génération au moment de la rédaction de ce document, dont Apertus — précisément le modèle open source suisse déjà mentionné dans la section sur le continuum d'ouverture.

Le vrai dilemme : large choix ou souveraineté ?



Rarement les deux à la fois.

FIG. 7 : Schéma en deux colonnes comparant Ollama (hébergé aux États-Unis, large catalogue de modèles open weight et open source, mais transfert de données hors UE au regard du RGPD) et Infomaniak (hébergé en Suisse, conforme RGPD par décision d'adéquation, mais catalogue plus restreint de huit modèles dont Apertus). Conclusion du schéma : rarement les deux avantages à la fois

Le dilemme est donc réel : un large choix de modèles (du côté d'Ollama) ou une réelle souveraineté des données (du côté d'Infomaniak) sont rarement obtenus à la fois. Ce dernier rebondissement rappelle une leçon plus large : **l'indépendance technique n'est pas la même chose que l'indépendance réelle**. Choisir un modèle ouvert est une première étape nécessaire, mais elle ne suffit pas : il faut aussi regarder l'infrastructure sur laquelle ce modèle est réellement exécuté.

Sources et vérifications

Toutes les sources ci-dessous ont été vérifiées par récupération réelle (URL consultée directement), pas reconstituées de mémoire.

Point de départ de la veille

- « De l'API fermée à l'open source » — vidéo d'Anaïs sur YouTube, qui introduit la distinction entre poids, code d'entraînement et données — le point de départ de cette veille, développé et complété ici avec l'angle réglementaire (AI Act), l'angle frugalité énergétique, et le twist final RGPD/Infomaniak.

Open Source AI Definition (OSAID) et logiciel libre

- Open Source AI Definition (OSAID) — Open Source Initiative
- OSI Open Source Definition
- Free Software Definition — Free Software Foundation
- Four Freedoms of Free Software — Richard Stallman, FSF
- Free Software Foundation — présentation de la FSF
- Open Source Initiative — historique et mission
- RFC — Request for Comments, IETF
- Standards et spécifications ouvertes — W3C, OASIS

AI Act et réglementation

- AI Act — Commission européenne
- AI Act, article 2(12) — exemption open source
- AI Act, article 51(2) — seuil de risque systémique (10^{25} FLOPs)
- Ce que les développeurs open source doivent savoir des règles de l’AI Act pour les modèles GPAI — Hugging Face — détaille ce dont l’open source est exempté (documentation technique de l’article 53(1)(a-b), représentant dans l’Union de l’article 54) et ce qui reste obligatoire (politique de droit d’auteur de l’article 53(1)(c), résumé du contenu d’entraînement de l’article 53(1)(d)), ainsi que les trois conditions d’éligibilité, dont la non-monetisation.
- Article 2 (champ d’application) — texte annoté de l’AI Act
- Spécifications et nombre de FLOP de Llama-3 405B — Meta

Frugalité énergétique et efficacité de l’inférence

- arXiv 2507.11417 — comparaison énergétique inférence locale vs API
- PUE (Power Usage Effectiveness) — efficacité des datacenters
- Optimisation de l’inférence par batching — Hugging Face
- Paradoxe de Jevons — efficacité économique et consommation
- Estimation de la consommation énergétique de GPT-4o — recherche OpenAI

Ollama — tarification et modèles

- Ollama Cloud — page tarifaire officielle
- Ollama Cloud Pricing 2026 : offres et heures de pointe
- Détail de tarification Ollama Cloud TPS
- Comparaison de coûts Ollama Cloud vs Claude et GPT (2026)

Infomaniak — alternative souveraine

- Infomaniak Trust Center — conformité RGPD et sécurité
- Infomaniak Developer Portal — documentation de l’endpoint de liste des modèles
- Liste des modèles disponibles au 8 septembre 2026 (source primaire, appel direct à l’API `GET https://api.infomaniak.com/2/ai/{id}/openai/v1/models`, et non la documentation) : huit modèles de génération — Ministral-3-14B, Qwen3.5-122B, Qwen3.5-397B, Gemma-4-31B, Kimi-K2.6, Nemotron-3-Nano-30B, Mistral-Small-4-119B, Apertus-v1.5-70B — ainsi que trois modèles d’embedding.
- Statut réglementaire de la Suisse : décision d’adéquation de la Commission européenne, la LPD (loi suisse sur la protection des données) révisée étant reconnue équivalente au RGPD depuis septembre 2023, ce qui dispense de clauses contractuelles types pour les transferts de données entre l’Union européenne et la Suisse.

Modèles d’IA mentionnés, par catégorie du continuum

Fermé / API : GPT (OpenAI), Claude (Anthropic), Gemini (Google), Amazon Nova (AWS).

Open weight : Llama (Meta), Mistral (Mistral AI), DeepSeek (DeepSeek), Qwen (Alibaba), Grok-1 (xAI).

Open weight++ : gpt-oss (OpenAI), Gemma (Google), Qwen (Alibaba, versions documentées).

Open source (poids, données et code publiés) : OLMo (Allen Institute for AI), Amber et Apertus (LLM360), Luciole (OpenLLM France), Nemotron (NVIDIA), Pythia (EleutherAI).